



Budget Bridge

Personal Finance Tracking · Budgeting Insights · AI Assistance

CS 451R | Spring 2026

Jacob Biddinger · Deming Lei · Brian Duong · Kevin Zhang

Team Introduction



Brian Duong

Project Manager / Documentation



Deming Lei

LLM Implementation Lead



Jacob Biddinger

Backend / Database



Kevin Zhang

Frontend / Backend / Auth

Project Overview

Budget Bridge is a full-stack personal finance web application that allows users to track income and expenses, view analytics on a dashboard, and interact with an AI financial assistant.

Problem Statement

Gap in the Market

- Many budgeting tools are complex or require paid subscriptions
- No easy way to get AI-powered financial insights from one place within a budgeting app
- Users struggle to understand spending patterns over time

Why It Matters

- Many Americans live paycheck to paycheck with little to no money left after covering basic expenses
- Many people don't track their monthly spending at all

Our Goal

- Provide users with a free and easy to use budgeting app that provides insights and assistance with an AI chatbot to help with budgeting finances

Solution Overview



Secure Auth

Register / login with email verification, hashed passwords, session management & password reset.



Expense Tracking

Log expenses with date, category, amount, and description. Filter, sort, and delete entries.



Dashboard Analytics

Visual charts: income vs expenses trend and income/spending by category



AI Assistant

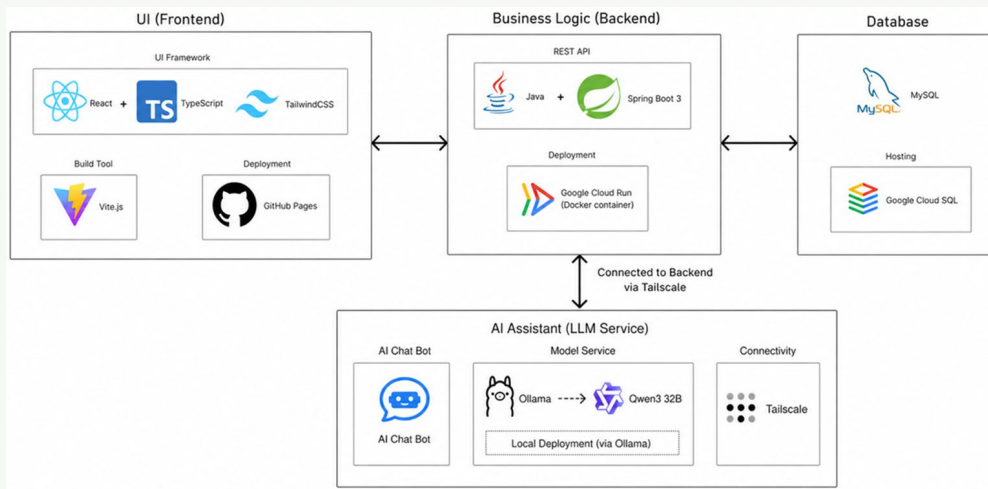
Floating chatbot powered by Ollama LLM providing personalized budget recommendations and financial insights



Income & History

Track income sources, view full transaction history, and undo accidental deletions.

Development Process and Tech Stack



We followed an agile-style and feature-driven development process. Each week, we selected a specific feature or issue, implemented it, tested it, and refined the application based on the results. This helped us gradually build the system while keeping the project manageable.

Tech Stack & Tools

Frontend: React · TailwindCSS

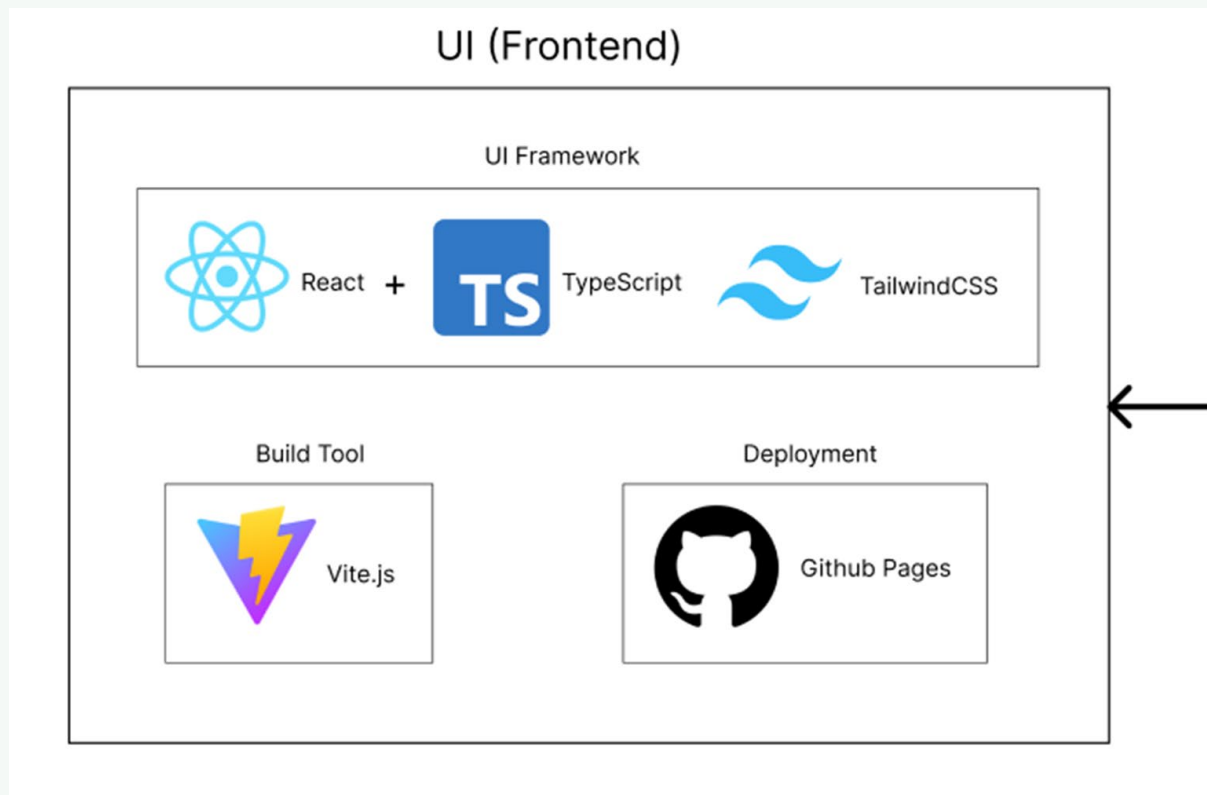
Backend: Spring Boot (Java) · JUnit 5 + Mockito

Database: MySQL · Hibernate / JPA

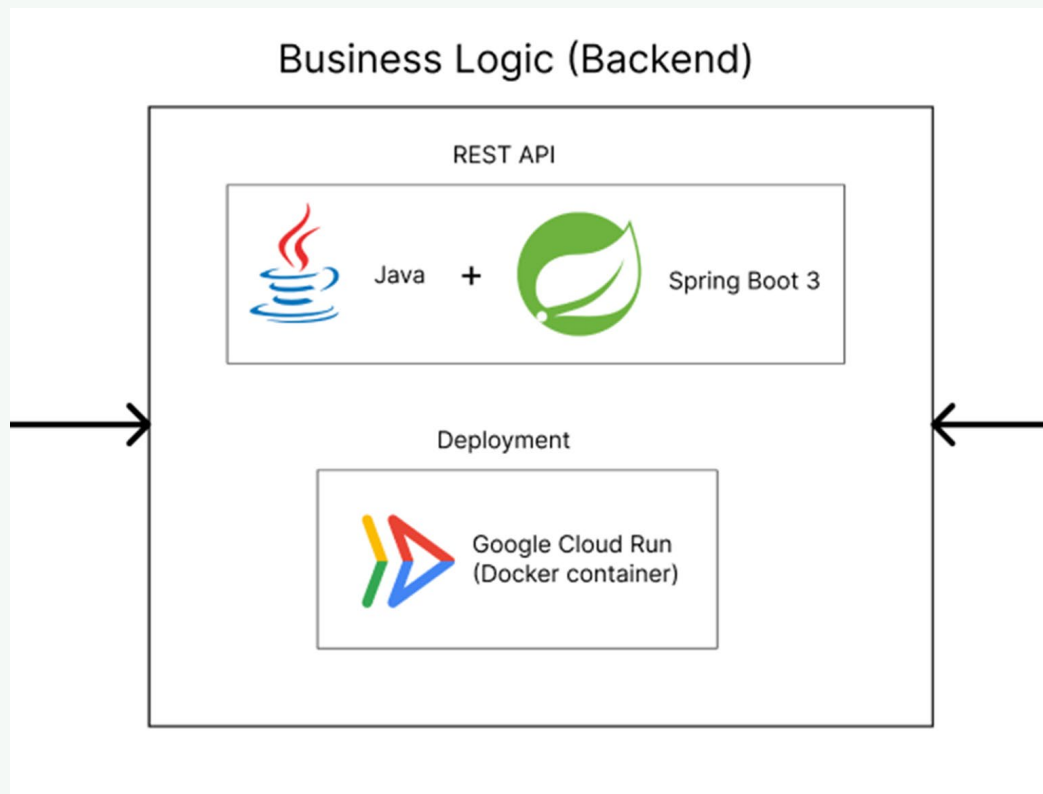
AI / LLM: Ollama (self-hosted) · Tailscale

DevOps: GitHub · JIRA · Google Cloud SQL + Run · GitHub Pages

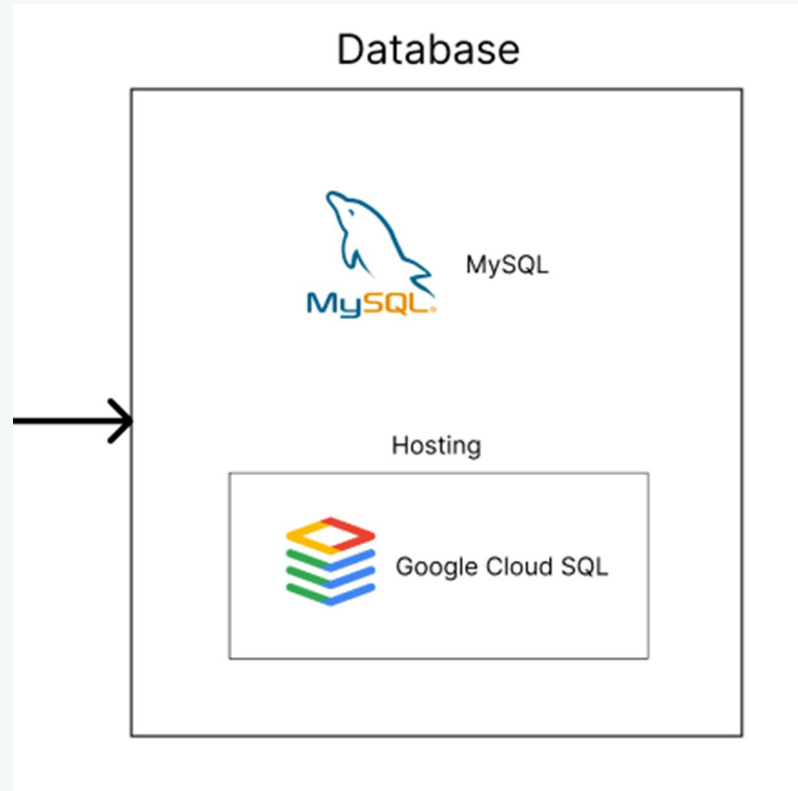
Frontend



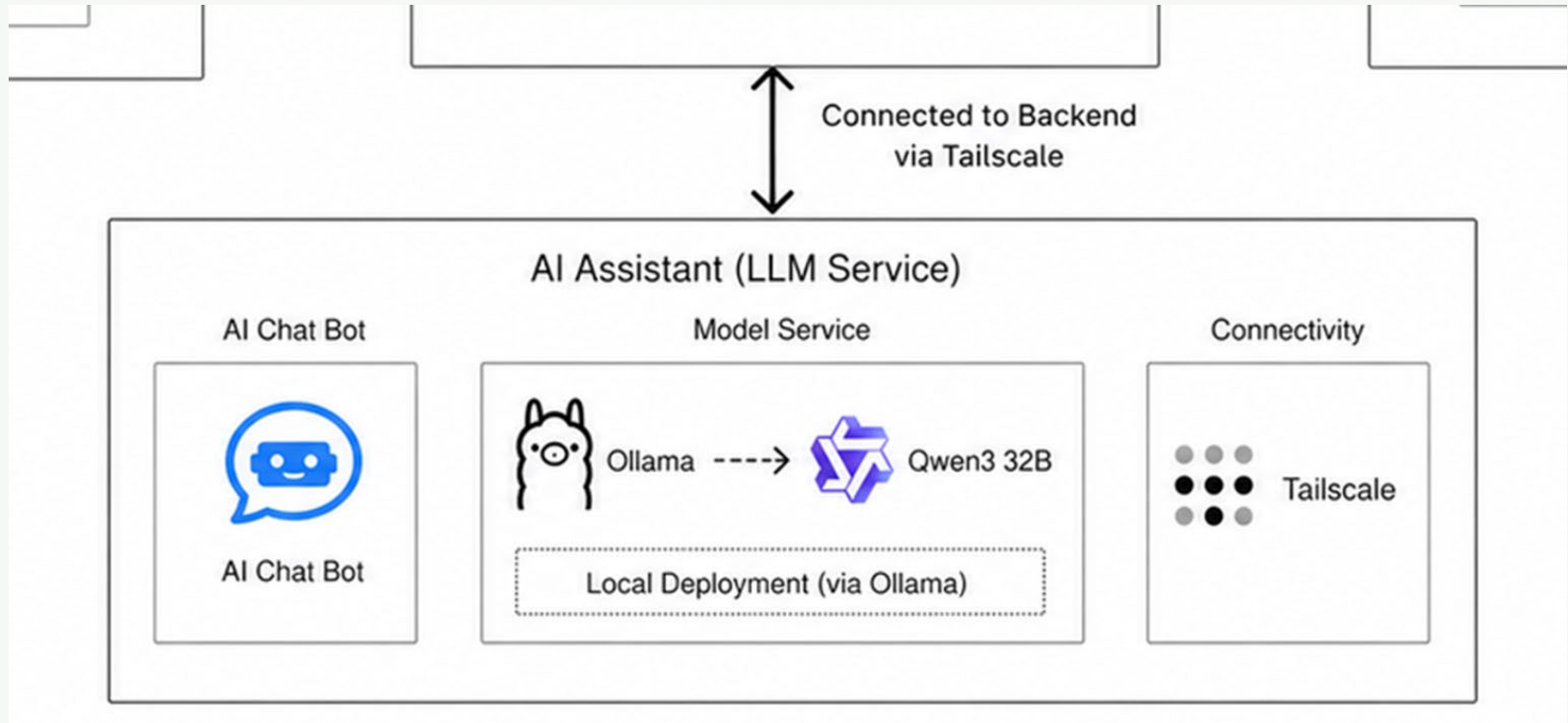
Backend



Database



AI Implementation



Challenges



Duplicate username registration showed no frontend error



Updated frontend handler to parse the 409 error response body from the backend and surface the message to the user.



Password reset link could be reused multiple times



Modified the `resetPassword` service to delete the token from the DB immediately after a successful reset.



Self-hosted LLM (Ollama) integration complexity



Isolated LLM calls to a dedicated service layer; provided context from live DB data so the assistant could give accurate recommendations.

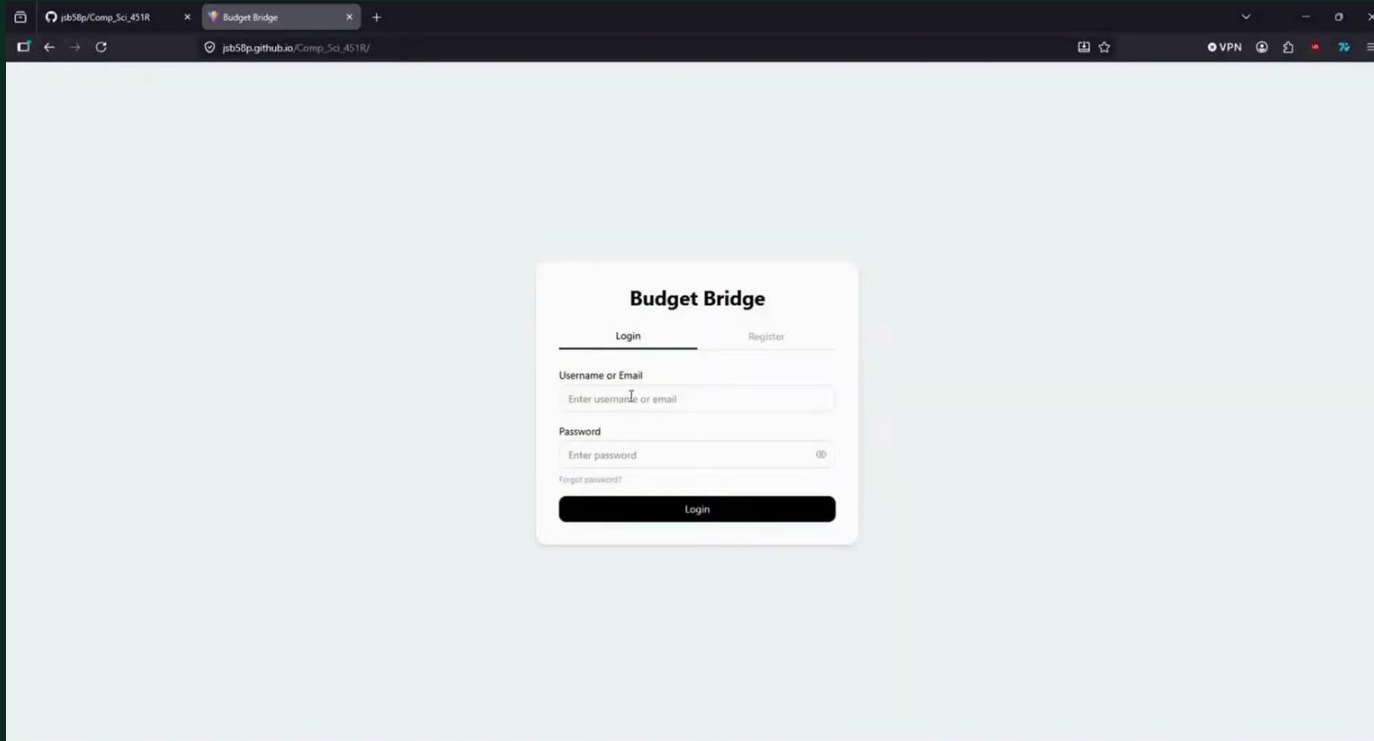


Self-hosted Ollama LLM was difficult to connect to the cloud-deployed project because the local server was outside the same LAN and blocked by firewall and network restrictions



Used Tailscale to place the cloud backend and local Ollama server in the same private tailnet, then updated firewall and Ollama host settings to allow secure LLM API access.

Demonstration



The screenshot shows a web browser window with two tabs. The active tab is titled "Budget Bridge" and shows a login page. The browser's address bar displays the URL "jsb58p.github.io/Comp_Sci_451R/". The login page has a title "Budget Bridge" and two tabs: "Login" (selected) and "Register". Below the tabs are two input fields: "Username or Email" with the placeholder text "Enter username or email" and "Password" with the placeholder text "Enter password". There is a "Forgot password?" link below the password field. At the bottom of the form is a black "Login" button.

Budget Bridge

Login Register

Username or Email
Enter username or email

Password
Enter password

[Forgot password?](#)

Login

Results and Impact

33

Tests

Automated unit tests passed

100%

Core

Requirements met

What Was Tested

UserService (Backend)

Registration, login, email verify, password reset

17 tests

SpendingController (Backend)

Auth checks, CRUD for expenses, 401/404/200 responses

10 tests

Frontend (Cypress)

DOM render, color change, accessibility, utility functions

6 tests

Requirements Met



Web Application

Frontend on Github Pages, Backend on Google Cloud Run, Database on Google Cloud SQL, no CMS.



Database

MySQL database with persistent storage via Hibernate/JPA and password hashing with BCrypt.



Unit testing

Automated unit testing using JUnit + Mockito



Frontend

Tailwind CSS framework + React



LLM Integration

AI Chatbot on self-hosted Ollama model via Tailscale



User Profile and Settings

Dark/Light mode with user profile allowing password change.



Home Page / Dashboard / Login / Register Page



Additional pages

Spending / Tracking / History page



Stretch Goal 1 (Create Web API layer for backend interactions)

Spring Boot API, frontend and backend are fully decoupled, communicates through REST endpoints



Stretch Goal 2 (Self-hosted authentication with password reset)

Email verification and password reset via emailed token link

Conclusion & Future Work

Key Achievements

- Full-stack personal finance budgeting web app
- All pages functional with live data
- AI financial assistant integrated with live account context
- Frontend and backend fully decoupled and deployed on Google Cloud Run + GitHub Pages
- Self-hosted authentication from scratch for email verification and password reset

Future Work and Future Considerations

- Bank account CSV import
- Mobile-responsive app
- Multi-user household budgeting
- Advanced ML spending predictions

Thank you Commerce Bank Team and Professor Shah for your support and guidance!